Practical:8

Date:8/10/2026

Hyperparameter Optimization – I

Perform hyperparameter optimizations by using Grid and Random search techniques for Regression and classification problem.

Procedure:

1. Import the required Python libraries, including NumPy and the necessary modules from Scikit-learn.

2. Load the Breast Cancer dataset for classification and the Diabetes dataset for regression.

3. Separate the independent variables X and dependent variable y for both datasets.

4. Split each dataset into training and testing sets using train_test_split(), with 80% of the data for training and 20% for testing.

5. Create a machine learning pipeline using StandardScaler() for feature scaling and LogisticRegression () for classification.

6. Apply Grid Search using GridSearchCV() to identify the best value of the regularization parameter C based on five-fold cross-validation and accuracy.

7. Apply Random Search using RandomizedSearchCV() to select parameter values randomly and identify the best classification model.

8. Evaluate both classification models using cross-validation accuracy and test accuracy.

9. Load the Diabetes dataset and create a pipeline using StandardScaler() and Ridge() regression.

10. Apply Grid Search and Random Search to find the best value of the regularization parameter , using five-fold cross-validation and mean absolute error (MAE).

11. Evaluate both regression models using cross-validation MAE and test MAE.

12. Display the best parameters and performance measures for Grid Search and Random Search for both classification and regression.

Solution:

```
# Grid Search and Random Search for Classification and Regression

import numpy as np

from sklearn.datasets import load_breast_cancer, load_diabetes
```

```
from sklearn.model_selection import train_test_split, GridSearchCV,
RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import accuracy_score, mean_absolute_error

# PART A: Classification
print("=== Classification ===")
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)
model = Pipeline([("scaler", StandardScaler()),
   ("logistic", LogisticRegression(max_iter=5000))])

# Grid Search
param_grid = {"logistic__C": [0.01, 0.1, 1, 10, 100]}
grid_search = GridSearchCV(estimator=model,param_grid=param_grid,
   cv=5,scoring="accuracy",n_jobs=-1)
grid_search.fit(X_train, y_train)
print("\nGrid Search")
print("Best Parameters:", grid_search.best_params_)
print("Best CV Accuracy:", grid_search.best_score_)
y_pred_grid = grid_search.predict(X_test)
print("Test Accuracy:", accuracy_score(y_test, y_pred_grid))
```

```
# Random Search
param_random = {"logistic__C": np.logspace(-5, 2, 20)}
random_search =
RandomizedSearchCV(estimator=model,param_distributions=param_random,
   n_iter=10,cv=5,scoring="accuracy",random_state=42,n_jobs=-1)
random_search.fit(X_train, y_train)
print("\nRandom Search")
print("Best Parameters:", random_search.best_params_)
print("Best CV Accuracy:", random_search.best_score_)
y_pred_random = random_search.predict(X_test)
print("Test Accuracy:", accuracy_score(y_test, y_pred_random))

# PART B: Regression
print("\n=== Regression ====")
data = load_diabetes()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42)
model = Pipeline([("scaler", StandardScaler()),("ridge", Ridge())])

# Grid Search
param_grid = {"ridge__alpha": [0.01, 0.1, 1, 10, 100]}
grid_search = GridSearchCV(estimator=model,param_grid=param_grid,
   cv=5,scoring="neg_mean_absolute_error",n_jobs=-1)
grid_search.fit(X_train, y_train)
print("\nGrid Search")
print("Best Parameters:", grid_search.best_params_)
print("Best CV MAE:", -grid_search.best_score_)
```

```
y_pred_grid = grid_search.predict(X_test)
print("Test MAE:", mean_absolute_error(y_test, y_pred_grid))

# Random Search
param_random = {"ridge__alpha": np.logspace(-3, 3, 20)}
random_search = RandomizedSearchCV(estimator=model,
   param_distributions=param_random,n_iter=10,cv=5,
   scoring="neg_mean_absolute_error",random_state=42,n_jobs=-1)
random_search.fit(X_train, y_train)
print("\nRandom Search")
print("Best Parameters:", random_search.best_params_)
print("Best CV MAE:", -random_search.best_score_)
y_pred_random = random_search.predict(X_test)
print("Test MAE:", mean_absolute_error(y_test, y_pred_random))
```

Output:

```
=== Classification ===
Grid Search
Best Parameters: {'logistic__C': 0.1}
Best CV Accuracy: 0.9802197802197803
Test Accuracy: 0.9736842105263158

Random Search
Best Parameters: {'logistic__C': np.float64(0.11288378916846883)}
Best CV Accuracy: 0.9802197802197803
Test Accuracy: 0.9736842105263158
```

=== Regression ====

Grid Search

Best Parameters: {'ridge__alpha': 10}

Best CV MAE: 45.401455944145546

Test MAE: 42.85682524780061

Random Search

Best Parameters: {'ridge__alpha': np.float64(2.976351441631316)}

Best CV MAE: 45.440051928254334

Test MAE: 42.828881923799166